

JavaFX事件处理机制

北京理工大学计算机学院
金旭亮

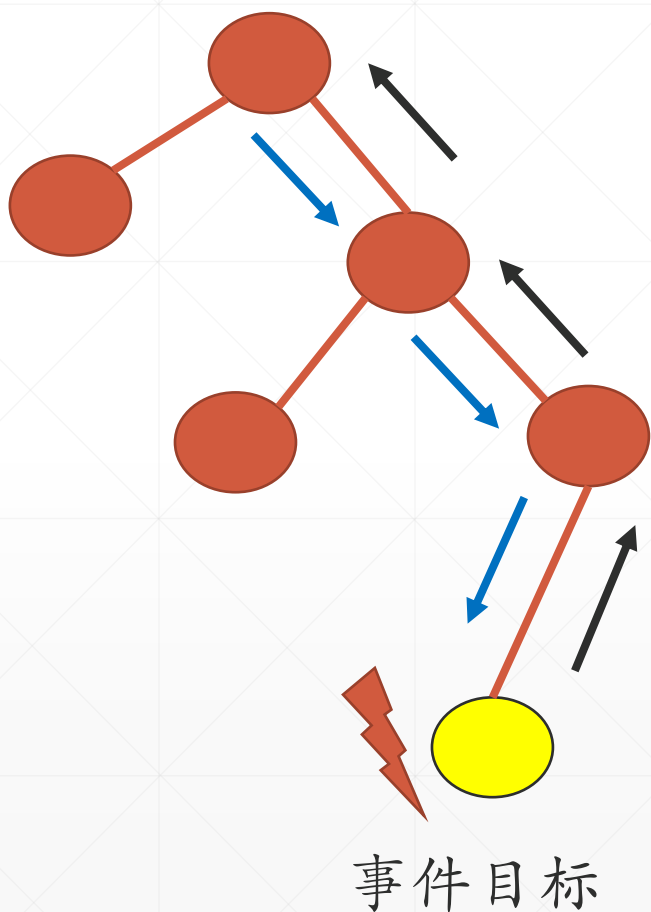


事件对象（Event类的实例）的属性

事件本身是javafx.event.Event类（或其子类的实例）的实例，它拥有以下三个属性：

属性	描述
事件类型 (Event Type)	哪种事件？ 是鼠标移动，还是键盘敲击？
源(Source)	事件的来源，表示该事件在事件派发链中的位置。事件通过派发链传递时，“源”会随之发生改变。
目标(Target)	触发了事件的Node节点，位于事件派发链的末尾。 “目标”不会改变，但是如果某个事件过滤器在事件捕获阶段消费了该事件，“目标”将不会收到该事件。

事件在控件树中的传播过程，称为“事件派发链”



事件派发链，实际上是一个双向链表，由事件目标对象负责创建，与特定的场景图相关联。

事件触发时，事件对象在链中传送。

事件响应方法（分为EventFilter和EventHandler两类）接收事件对象作为参数。

→ 事件捕获，调用EventFilter

→ 事件冒泡，调用EventHandler



在事件传播的过程（称为“事件派发链”）中.....

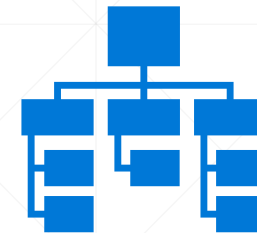


EventType和EventTarget是不变的



EventSource则会随着事件的传播过程，动态地改变，它的当前值，表明此事件当前“传播”到了哪里（哪个Node对象）

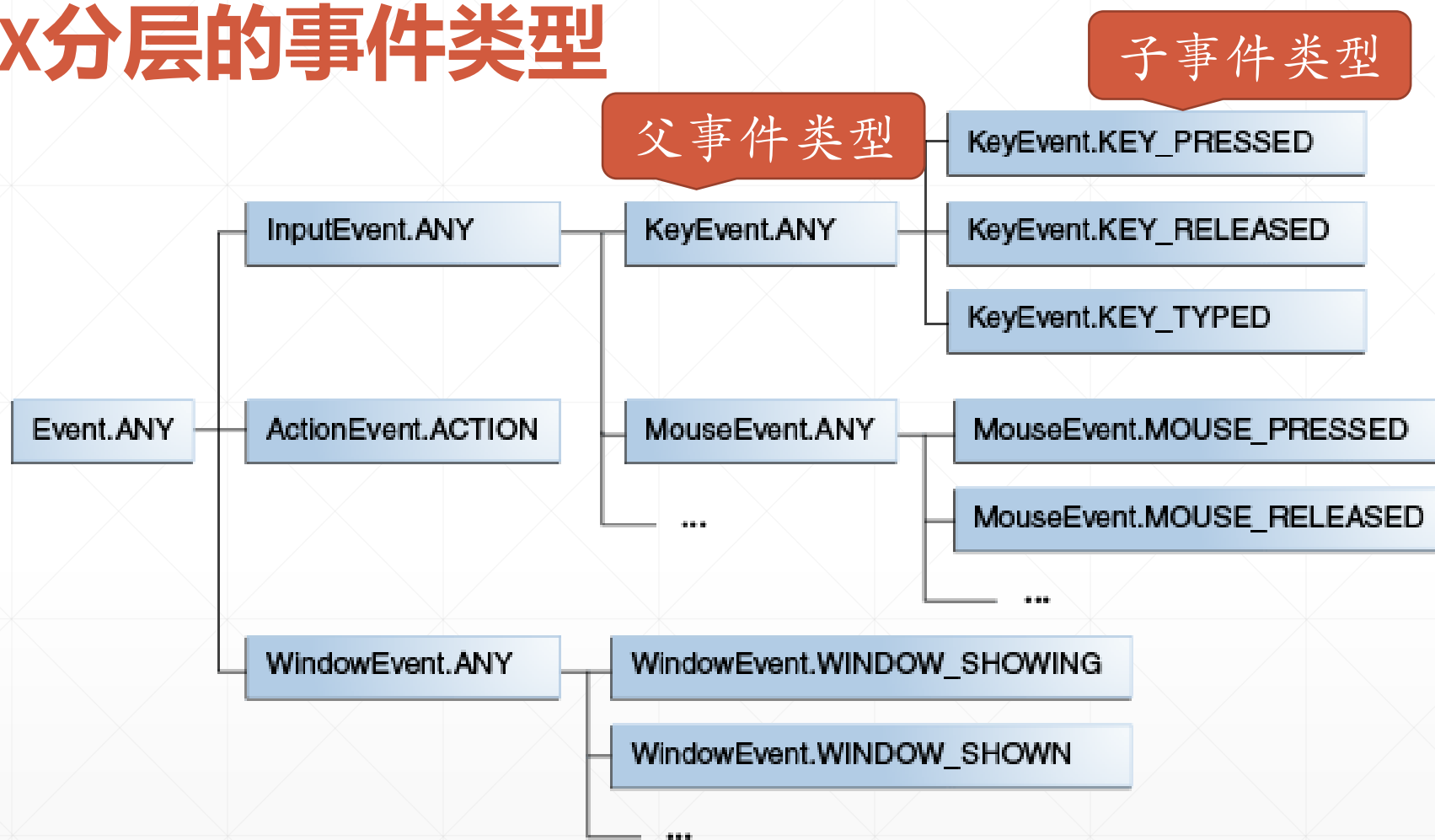
事件类型



“**事件类型(Event Type)**”是EventType类的实例。JavaFX对单个事件类的多种事件进行了细化归类。例如，KeyEvent类包含如下事件类型：

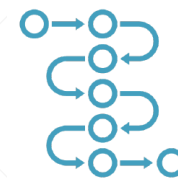
- ▶ KEY_PRESSED: 按键按下事件
- ▶ KEY_RELEASED: 按键松开事件
- ▶ KEY_TYPED: 按键事件（不细分按下和松开动作）

JavaFX分层的事件类型



一个响应父事件的EventHandler或EventFilter，也会响应其子事件。

事件分发流程



● 选择目标

当一个事件发生时，系统根据内部规则决定哪一个**Node**是事件目标

● 构造路径

事件路径是由“**事件派发链 (event dispatch chain)**”决定的，此链是在被选中的事件目标的 `buildEventDispatchChain` 方法实现中创建的。

● 捕获事件

调用**EventFilter**，如果事件被特定Filter所“**消费 (Consume)**”，则中止事件的传播，后面的Filter和Handler不会再被调用

● 事件冒泡

调用**EventHandler**，如果事件被特定Handler所消费，中止事件的进一步传播，后面的Handler不会再被调用

事件处理函数: EventFilter



Event Filter在事件捕获阶段执行。父节点的EventFilter可以为多个子节点提供公共的事件处理，并且如果需要的话，也可以消费掉事件以阻止子节点收到该事件。当某事件被传递并经过注册了Event Filter的Node节点时，对应事件类型的Event Filter（如果有的话）就会被执行。



一个节点可以注册多个Event Filter。Event Filter执行的顺序取决于事件类型的层级关系。特定事件类型的Event Filter会先于通用事件类型的Event Filter执行。例如，MouseEvent.MOUSE_PRESSED事件的Event Filter会在InputEvent.ANY事件的Event Filter之前执行。属于同一层级的Event Filter，其执行顺序并未指定。

事件处理函数：EventHandler



Event Handler在事件冒泡阶段执行。当某事件返回并经过注册了Event Handler的节点时，为该事件类型注册的Event Handler就会被执行。如果子节点的Event Handler未消费掉对应的事件，那么父节点的Event Handler就可以在子节点处理完成以后来处理该事件，并且它还可以为多个子节点提供公共的事件处理服务。



一个节点可以注册多个Event Handler。Event Handler执行的顺序取决于事件类型的层级。特定事件类型的Event Handler会先于通用事件类型的Event Handler执行。例如，KeyEvent.KEY_TYPED事件的EventHandler会在InputEvent.ANY事件的EventHandler之前执行。

事件的消费

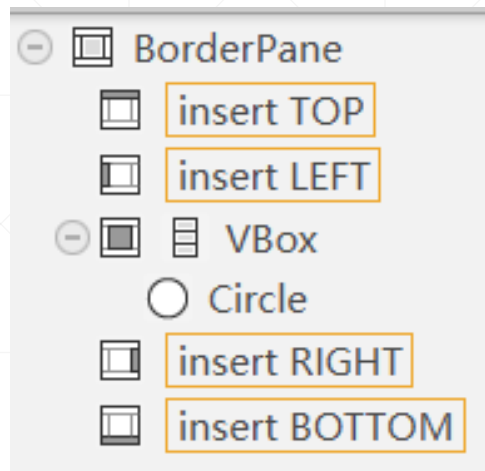
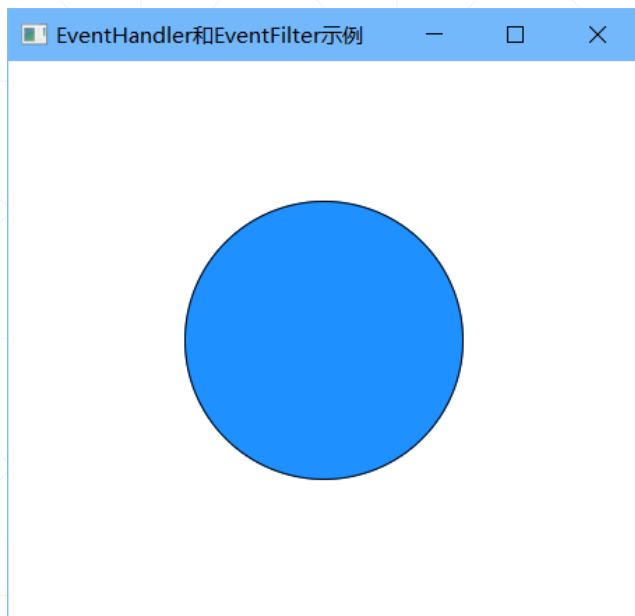


事件可以被Event Filter或 Event Handler在事件派发链中的任意节点上通过调用**consume()**方法消费掉。此方法表示事件的处理已经完成，后继传播就没有必要了。



在Event Filter中消费掉对应的事件会阻止事件派发链上的任何子节点再响应该事件。对应地，在Event Handler中消费掉对应的事件会阻止事件派发链中的任何父Event Handler再处理该事件。但是，如果消费掉该事件的节点为该事件注册了多个Event Filter或者Event Handler，这些同级的Event Filter或者Event Handler仍然会被执行。

事件示例: EventHandlerAndFilterDemo



示例中包容一个三级节点树，点击圆，
可以在控制台窗口中看到输出

```
EventHandlerAndFilterApplication x
"C:\Program Files\Java\jdk-19\bin\java.exe" ...
BorderPaneEventFilter:EventType:MOUSE_PRESSED Source:BorderPane Target:Circle isConsumed:false
VBoxEventFilter:EventType:MOUSE_PRESSED Source:VBox Target:Circle isConsumed:false
CircleEventFilter:EventType:MOUSE_PRESSED Source:Circle Target:Circle isConsumed:false
CircleEventHandler:EventType:MOUSE_PRESSED Source:Circle Target:Circle isConsumed:false
VBoxEventHandler:EventType:MOUSE_PRESSED Source:VBox Target:Circle isConsumed:false
BorderPaneEventHandler:EventType:MOUSE_PRESSED Source:BorderPane Target:Circle isConsumed:false
```

示例分析

```
public class MainWindowController {  
    //注入视图中的三个控件  
    @FXML BorderPane borderPane;  
    @FXML VBox vBox;  
    @FXML Circle circle;  
  
    //统一的事件处理方法，输出event对象的相关属性值  
    //由参数决定是否消费此事件  
    private void handleAndConsumed(Event event, boolean consume, String message)  
    {  
        if(consume){  
            event.consume();  
        }  
        String info=String.format(  
            "EventType:%s Source:%s Target:%s isConsumed:%b",  
            event.getEventType(),  
            event.getSource().getClass().getSimpleName(),  
            event.getTarget().getClass().getSimpleName(),  
            event.isConsumed());  
        System.out.println(message+": "+info);  
    }  
}
```

//为三个节点附加EventFilter和EventHandler

//通过修改handleAndConsumed()方法的参数值，可以观察事件派发链的实际运行原理

```
public void initListener() {
```

```
    borderPane.addEventFilter(MouseEvent.MOUSE_PRESSED,  
        event->handleAndConsumed(event, false, "BorderPaneEventFilter"));  
    borderPane.addEventHandler(MouseEvent.MOUSE_PRESSED,  
        event->handleAndConsumed(event, false, "BorderPaneEventHandler"));
```

```
    vBox.addEventFilter(MouseEvent.MOUSE_PRESSED,  
        event->handleAndConsumed(event, false, "VBoxEventFilter"));  
    vBox.addEventHandler(MouseEvent.MOUSE_PRESSED,  
        event->handleAndConsumed(event, false, "VBoxEventHandler"));
```

```
    circle.addEventFilter(MouseEvent.MOUSE_PRESSED,  
        event->handleAndConsumed(event, false, "CircleEventFilter"));  
    circle.addEventHandler(MouseEvent.MOUSE_PRESSED,  
        event->handleAndConsumed(event, false, "CircleEventHandler"));
```

//如果想测试响应父事件，取消以下代码的注释

```
//    borderPane.addEventFilter(MouseEvent.ANY,  
//        event->handleAndConsumed(event, true, "BorderPaneEventFilter"));
```

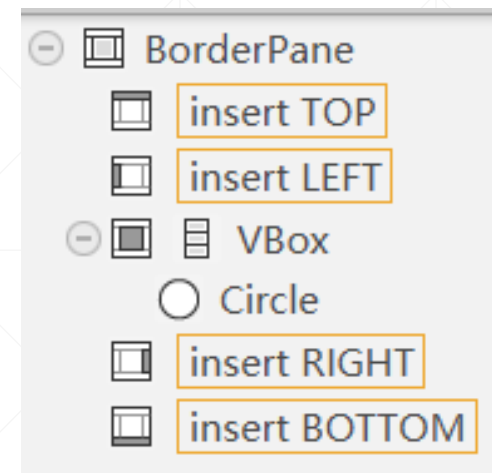
```
//    circle.addEventHandler(MouseEvent.ANY,  
//        event->handleAndConsumed(event, true, "CircleEventHandler"));
```

```
}
```

一些测试结果

示例运行后，用鼠标点击圆，在控制台窗口中观察输出，会发现：

默认情况下，先调用Filter，后调用Handler，注意调用顺序。

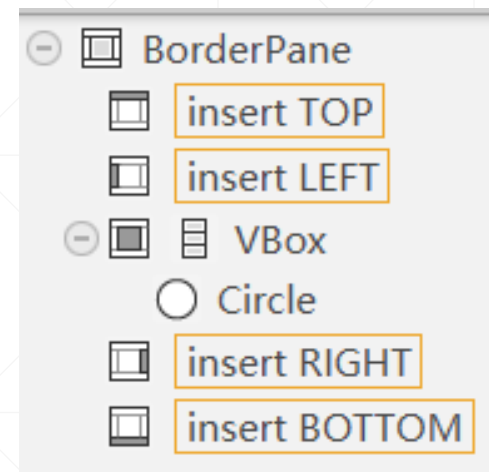


```
EventHandlerAndFilterApplication x
"C:\Program Files\Java\jdk-19\bin\java.exe" ...
BorderPaneEventFilter:EventType:MOUSE_PRESSED Source:BorderPane Target:Circle isConsumed:false
VBoxEventFilter:EventType:MOUSE_PRESSED Source:VBox Target:Circle isConsumed:false
CircleEventFilter:EventType:MOUSE_PRESSED Source:Circle Target:Circle isConsumed:false
CircleEventHandler:EventType:MOUSE_PRESSED Source:Circle Target:Circle isConsumed:false
VBoxEventHandler:EventType:MOUSE_PRESSED Source:VBox Target:Circle isConsumed:false
BorderPaneEventHandler:EventType:MOUSE_PRESSED Source:BorderPane Target:Circle isConsumed:false
```

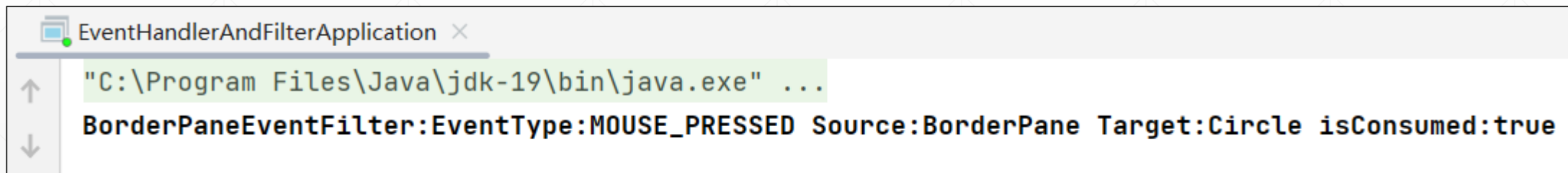
另外，注意Target始终是Circle，而Source则表明了“事件”“当前”传输到了哪里。

一些测试结果

```
public void initListener() {  
  
    borderPane.addEventFilter(MouseEvent.MOUSE_PRESSED,  
        event->handleAndConsumed(event, true, "BorderPaneEventFilter")  
    borderPane.addEventHandler(MouseEvent.MOUSE_PRESSED,  
        event->handleAndConsumed(event, false, "BorderPaneEventHandler"
```



在最顶层的BorderPane的EventFilter内，Consume掉事件，运行结果是：



所有下层控件的EventFilter和EventHandler都不会被调用。

请用类似的方式修改示例代码，了解JavaFX的事件派发链的运行机制。

挂接事件响应代码的方式



有两种方法为JavaFX Node对象挂接事件响应代码：

1

使用`addEventFilter()`、`addEventHandler()`

2

使用特定控件的`setOnXXX()`方法。

使用第一种方法，可以挂接多个事件响应方法，而第二种方法，只能挂接一个。

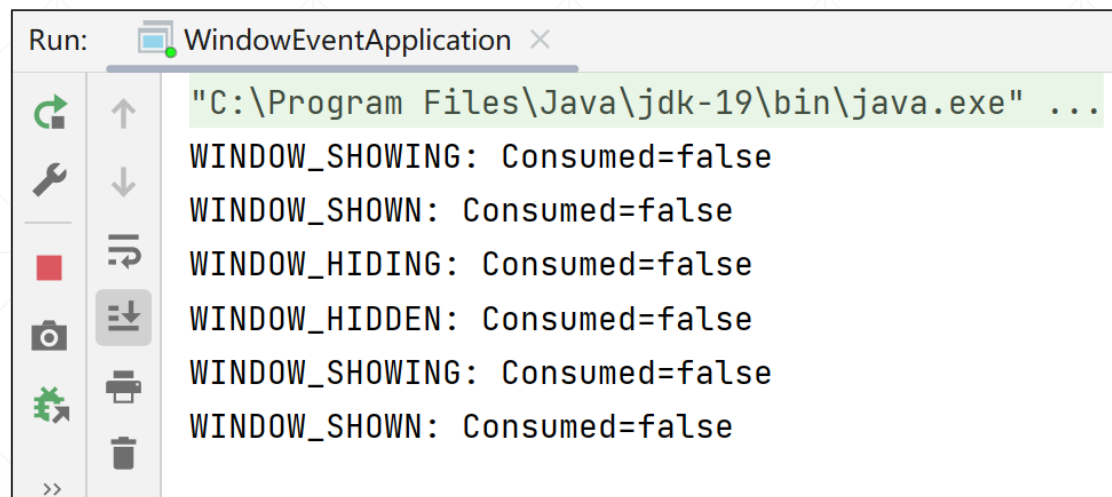
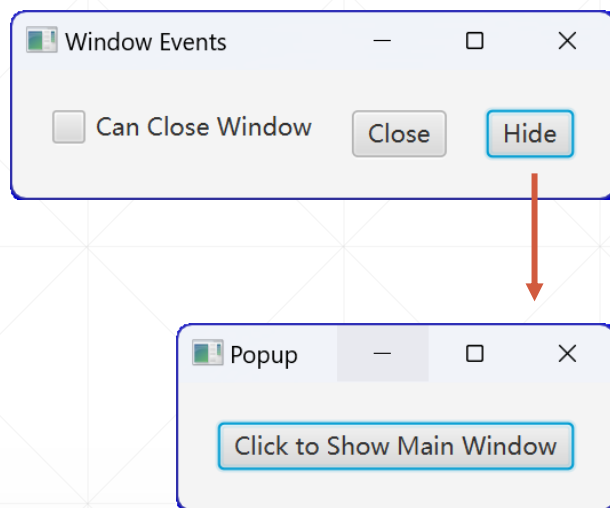
如果需要的话，可以调用`removeEventFilter()`和`removeEventHandler()`方法来移除事件监听器对象。

响应Windows事件

JavaFX为Stage提供的“窗体”事件列表：

Constant	Description
ANY	It is the supertype of all other window event types.
WINDOW_SHOWING	It occurs just before the window is shown.
WINDOW_SHOWN	It occurs just after the window is shown.
WINDOW_HIDING	It occurs just before the window is hidden.
WINDOW_HIDDEN	It occurs just after the window is hidden.
WINDOW_CLOSE_REQUEST	It occurs when there is an external request to close this window.

示例运行截图

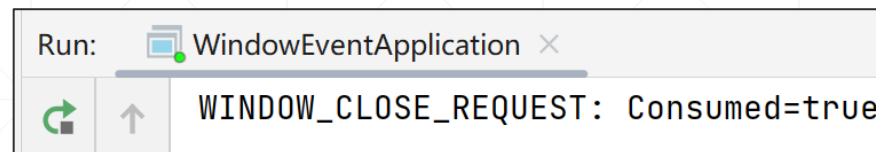
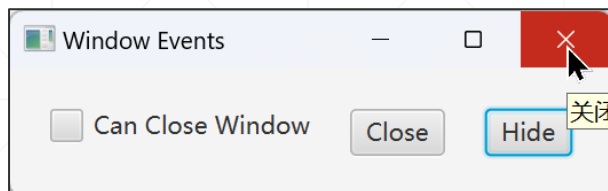


- 点击主窗体的“Hide”按钮，主窗体隐藏，Popup窗体出现，这时可以看到主窗体的HIDING和HIDDEN事件触发。
- 点击Popup窗体的“Click to Show Main Window”按钮，主窗体重新出现，其SHOWING和SHOWN事件再次触发。

阻止窗体的关闭



通过consume掉WINDOW_CLOSE_REQUEST事件，可以阻止用户点击“窗体”的关闭按钮直接关闭窗口。



选中“Can Close Window”选项后，WINDOW_CLOSE_REQUEST事件不被consume，窗体可被关闭。

